Introduction à CUDA

Ressources :

https://docs.nvidia.com/cuda/cuda-c-programming-guide/#

https://tcuvelier.developpez.com/tutoriels/gpgpu/cuda/introduction/?page=conclusions

Table des matières

1. **Introduction et historique**
2. **Programmation avec CUDA**
3. **Le modèle Logique**
4. **Le modèle Physique**

1. Introduction et Historique

Note : ce cours représente une brève introduction à CUDA. On pourra lire les différents ouvrages traitant du sujet [1, 2, 3] pour se familiariser avec les fonctionnalités avancées de CUDA.



CUDA is a parallel computing platform and programming model invented by NVIDIA. It enables dramatic increases in computing performance by harnessing the power of the graphics processing unit (GPU). NVIDIA provides a complete toolkit for programming on the CUDA architecture, supporting standard computing languages such as C, C++ and Fortran (NVidia)

1.1. Programmation Multi-core et many-core

1.1.1. L'ère du multi-core

Depuis l'invention du microprocesseur (**CPU = Central Processing Unit**) par Intel en 1971, les circuits intégrés ont connu de nombreuses améliorations qui ont concourru à l'augmentation de leurs performances :

- augmentation de la fréquence
- niveaux et taille de cache (L1, L2, L3, L4=eDRAM)
- exécution super scalaire
- circuits dédiés : FPU, SSE, AVX
- multi-core : dual, tri, quad, hexa, octo, ...

Cependant, en terme de parallèlisme, même si on dispose de plusieurs coeurs, ceux-ci exécutent en général des programmes différents, on est dans un modèle **MIMD** (*Multiple Instructions, Multiple Data*). Seules les unités dédiées[1] (SSE, AVX) permettent de faire du calcul parallèle, mais on parle ici de circuits vectoriels capables de traiter 4 réels simple précision dans le cas du SSE4.2.

[1] SSE, Streaming SIMD Extensions (SSE2, SSE3, SSE4 en 2008) : par exemple instructions pour additionner et multiplier plusieurs valeurs stockées dans un seul registre (registre SSE).

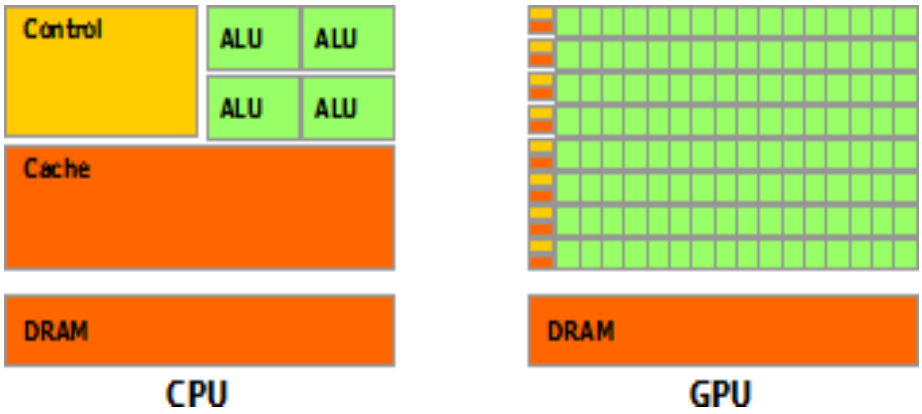
L'utilisation de plusieurs coeurs de microprocesseurs, ou de plusieurs mircroprocesseurs pour la programmation en parallèle est qualifiée de programmation **multi-core**.

Exemple d'application maximum de parcimonie[2]

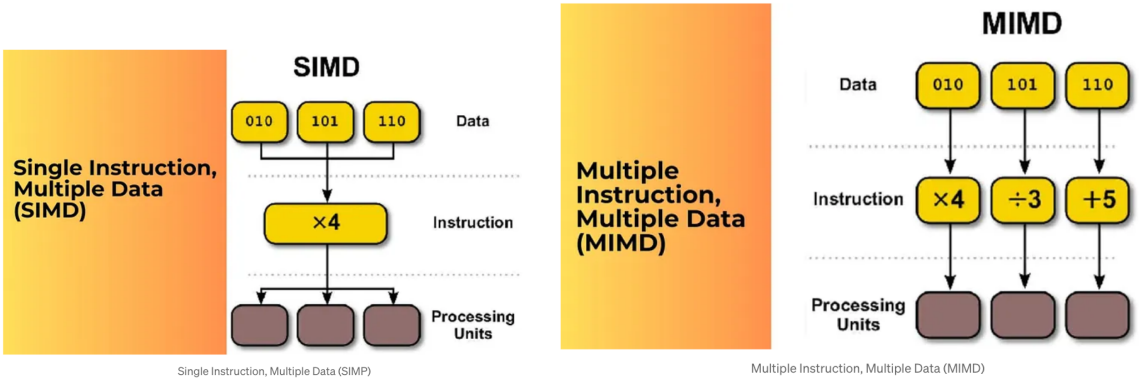
1.1.2. L'ère du many-core

En parallèle des CPU, les **GPU** (**Graphics Processing Unit**) circuits initialement conçus pour l'affichage graphique (notamment la 2D et dans une plus large mesure la 3D), ont connu un essor fulgurant. La rivalité des constructeurs **NVidia** et **ATI** (racheté par **AMD**) a été bénéfique et elle a mené à l'évolution des GPU vers des circuits dédiés au **calcul hautement parallèle**. En fonction du GPU on est capable de traiter plusieurs centaines ou milliers de réelles simples précisions en parallèle.

Comparaison multi et many core



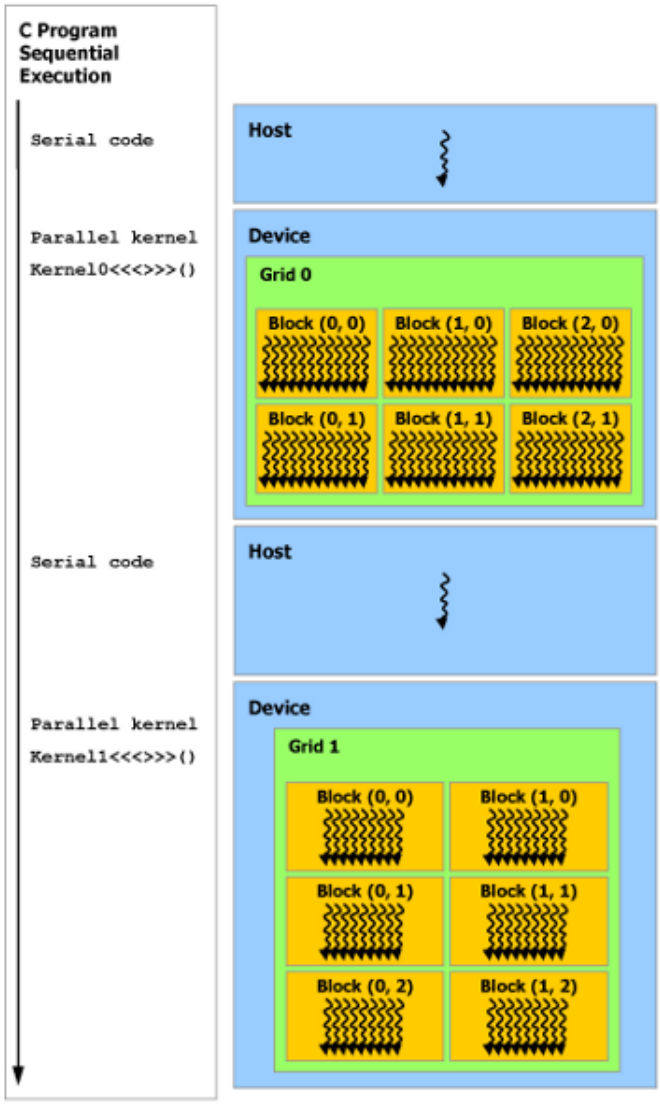
L'utilisation d'un GPU pour le calcul parallèle est qualifiée de programmation **many-core** et correspond à du **SIMD** (Single Instruction, Multiple Data) ou plutôt à ce que les constructeurs appellent du **SIMT** (Single Instruction, Multiple Thread).



Single Instruction, Multiple Data (SIMP) Multiple Instruction, Multiple Data (MIMD)

Source : https://afzalbadshah.medium.com/parallel-programming-models-simd-and-mimd-b2a5556d7efe

2 http://www.info.univ-angers.fr/~richer/ens_inra_crs3_parallel.php



Programmation hétérogène

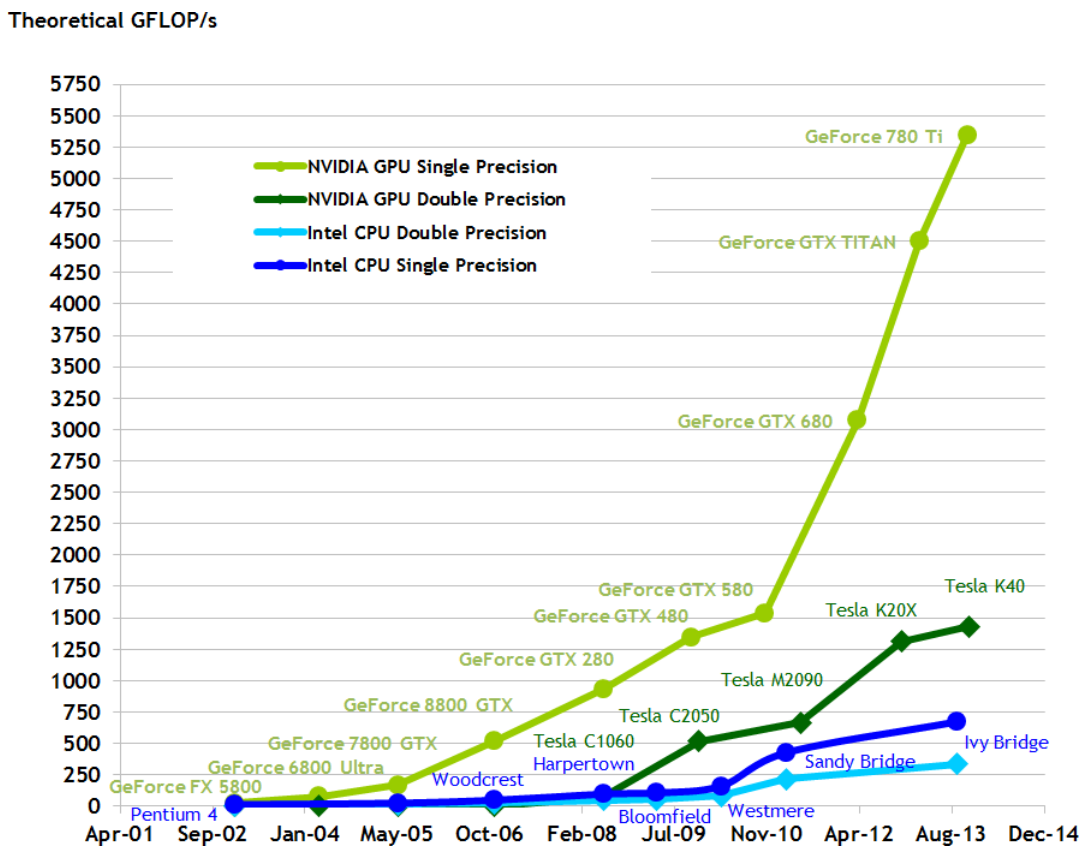
Source https://docs.nvidia.com/cuda/cuda-c-programming-guide/#

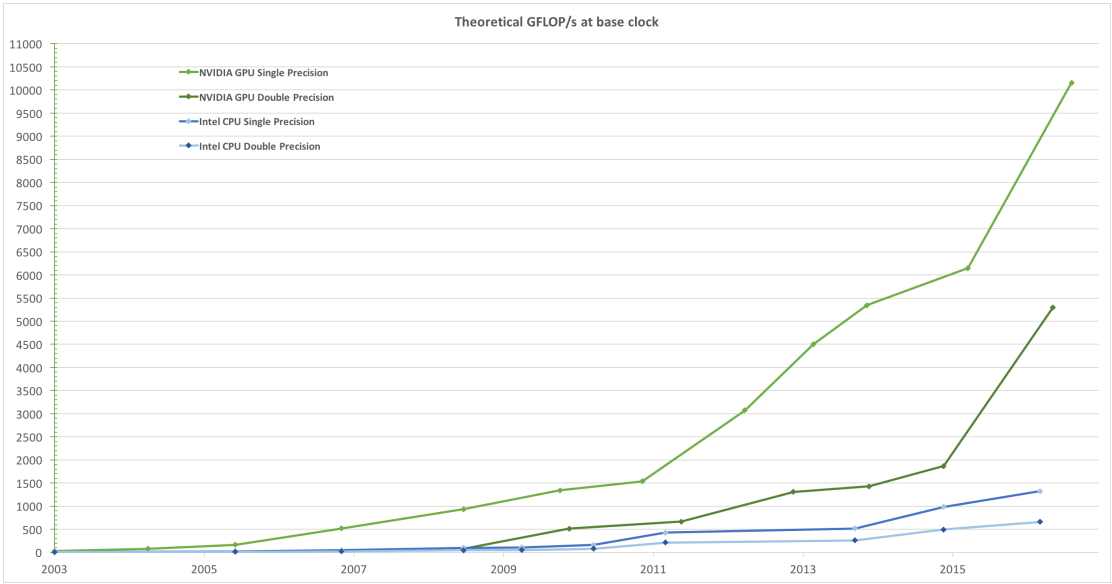
Notons qu'en 2009, la puissance de calcul d'un CPU était d'environ 100 GFlops alors que celle d'un GPU était 1 TFlops, soit 10 fois plus importante.

Les graphiques suivants peuvent être trouvés ici pour des versions plus récentes.

Evolution des performances en GFlops des CPU et GPU

(Source: Nvidia CUDA Programming Guide)



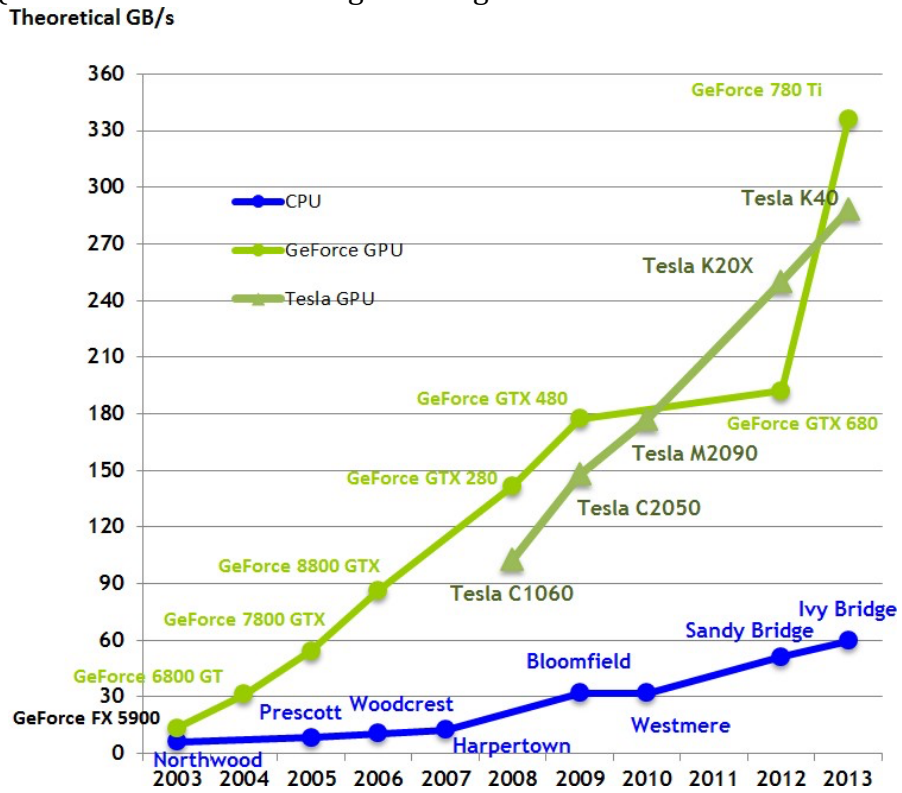


En 2013, un ordinateur personnel peut développer une puissance d'environ 200 gigaFLOPS avec un microprocesseur comme l ᵉ Intel Core i7-3770, de puissance comparable aux superordinateurs de 1995, et 5 621 gigaFLOPS avec un processeur graphique comme le Nvidia GTX 690, soit une puissance comparable aux superordinateurs de 2001 (*Top500 List - November 2001*, sur *top500.org*).
Le 28 janvier 2014, la plateforme de calcul distribué BOINC totalise 8,251 pétaFLOPS (10^{15}).

En 2015, le gouvernement américain lance un projet pour franchir la barre de l'exaFLOPS (10^{18} FLOPS).

Evolution de la Bande passante mémoire en Go/s des CPU et GPU

(Source: Nvidia CUDA Programming Guide



Comparaison CPU vs. GPU		
	CPU	GPU
concepteur	Intel	NVIDIA
modèle	Core i7 920	GTX 280
date de lancement	Juin 2008	Novembre 2008
prix (dollars)	260	260
fréquence	2.66 GHz	1.3 GHz
cores	4	240
threads	8	23040
GFlops	64	933
BP mémoire Go/s	31.8	141.7
TDP	130W	236W
GFlops/W	0.49	3.95

Dans le tableau comparatif précédent, si on désire obtenir la même puissance de calcul qu'une carte graphique grâce à un CPU, on devra utiliser 933/64=14.57 soit environ 15 CPU, ce qui représente :

- un coût d'achat de 15×260=3900 dollars (×15)
- et une consommation électrique de 15×130=1950 Watts (×8.26)

2. Programmer avec CUDA

2.1. Introduction

Jusqu'en 2006, les GPU étaient difficiles à programmer car il fallait utiliser les **API** (*Application Programming Interface*) des fonctions graphiques (OpenGL et Direct3D) ce qui limitait les performances et le type d'application parallélisable.

A partir de 2007, NVidia met à disposition des programmeurs : **CUDA** (*Compute Unified Device Architecture*)

> *"CUDA is a parallel computing platform and programming model invented by NVIDIA. It enables dramatic increases in computing performance by harnessing* the power of the graphics processing unit (GPU)." (NVidia)*
>
> ** to harness = exploiter*

La programmation en CUDA repose sur deux modèles :

- le **modèle logique** qui permet d'exprimer :
  - le calcul à réaliser (Code C/C++)
  - l'organisation des threads sous forme d'une grille de blocs de threads
- le **modèle physique** qui se charge de répartir et exécuter les threads sur les coeurs d'exécution du GPU

  Le problème majeur que l'on rencontre en CUDA est que les modèles logique et physique sont fortement couplés et notamment le modèle physique influe de manière importante sur l'organisation des threads

Trouver la solution la plus efficace pour résoudre un problème donné n'est pas simple comme nous le verrons. Il faut prendre en compte :

- choix de la taille de la grille et des blocs
- utilisation de la mémoire partagée (shared memory), de la mémoire constante, de la mémoire de textures
- éviter les branchements
- ...

2.2. Spécificités

Le GPU est en fait un véritable ordinateur miniature car il possède de nombreux coeurs de calcul ainsi que de la mémoire globale, de la mémoire cache (L1, L2), de la mémoire partagée.

Mémoire globale

La mémoire globale ne possède aucun cache, il est donc très important de suivre le mode d'accès aux données prévu pour maximiser la bande passante.

Ce mode d'accès est conçu pour permettre à l'ensemble des threads d'un half-warp (la moitié d'un warp) d'atteindre la mémoire globale en une seule transaction mémoire. Pour ce faire, ces accès mémoire doivent respecter certaines règles.
Ainsi il est notamment nécessaire que tous les threads d'un half-warp accèdent à des données

résidant dans un segment (zone d'éléments mémoire contigus) de taille :
- 32 octets si ces threads accèdent à des mots de 8 bits,
- 64 octets si ces threads accèdent à des mots de 16 bits,
- 128 octets si ces threads accèdent à des mots de 32 ou 64 bits

Si en revanche, le half-warp accède à des mots dans n segments différents (accès non contigus), n transactions mémoires seront réalisées. A noter que les éléments d'un segment auquel aucun thread du half-warp n'accède sont tout de même lus.

Mémoire locale

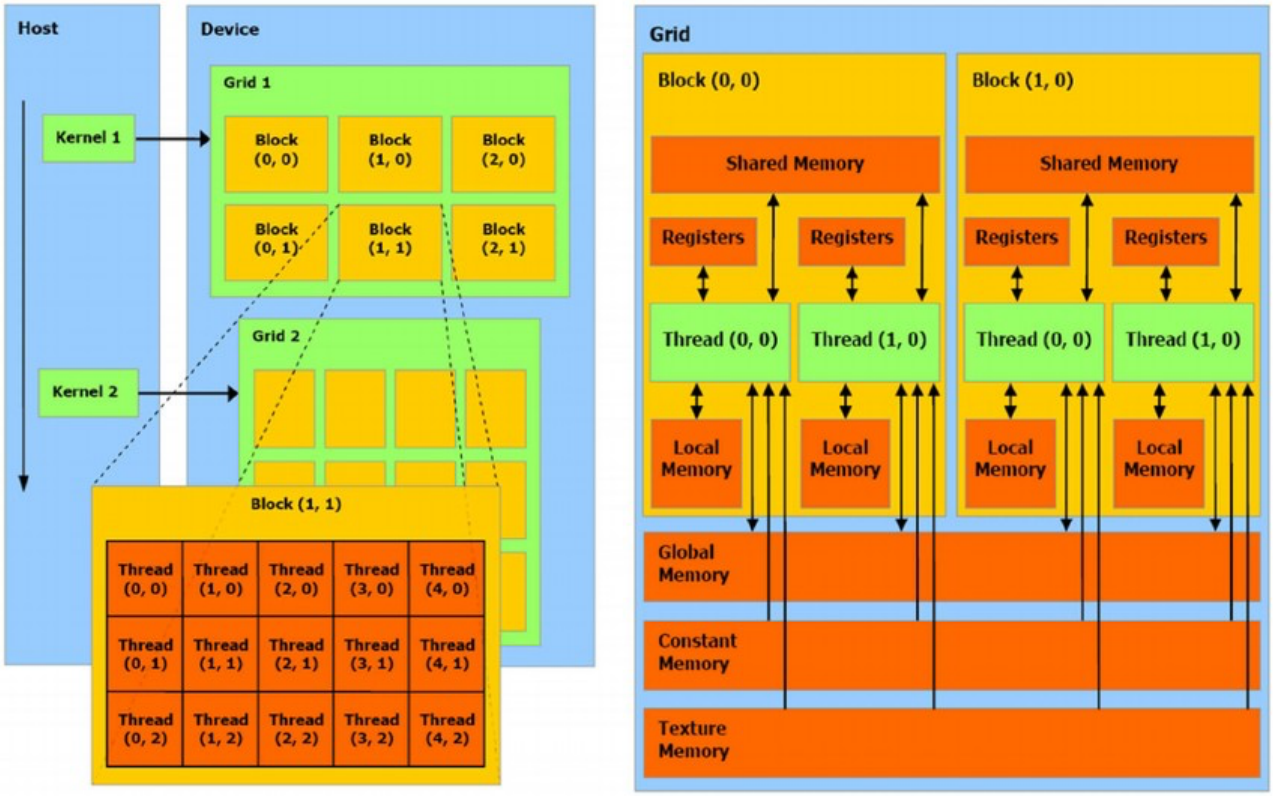
L'accès à la mémoire locale n'intervient que lorsque des variables automatiques sont affectées non pas aux registres par manque de ressources, mais à la mémoire dite locale.
Tout comme la mémoire globale, la mémoire locale ne possède pas de cache, les accès sont donc aussi coûteux que pour la mémoire globale.

Mémoire constante

A la différence des deux mémoires précédentes, la mémoire constante possèdent un cache. Par conséquent, pour tous les threads d'un half-warp, lire des données situées dans le cache de la mémoire constante est aussi rapide que pour des données situées dans des registres tant que tous ces threads lisent à la même adresse. Les coûts d'accès augmentent linéairement avec le nombre d'adresses distinctes accédées par les threads.

Mémoire partagée

La mémoire partagée est définitivement plus rapide que les mémoires globale ou locale. En fait, pour chaque thread d'un warp, accéder à la mémoire partagée est tout aussi rapide que d'accéder aux registres sous réserve cependant que ces accès mémoires se fassent sans conflit.



On appelle **host** l'ordinateur qui accueille le GPU.

Le GPU est quant à lui qualifié de **device**.

Lorsque l'on programme en CUDA on tente de différencier les données en mémoire centrale et celles en mémoire du GPU. On utilise en général un préfixe pour les noms de variables :

- **host = CPU** : h_mem, hMem, c_mem, CMem, ...
- **device = GPU** : d_mem, dMem, g_mem, gMem, ...

Pour que le GPU puisse réaliser les calculs parallèles, il faut créer une copie des données de la mémoire centrale de l'host vers la mémoire globale du GPU (device) (*Excepté à partir de CUDA 6 qui profite d'un support pour la mémoire unifiée mais en architecture 64 bits*). On dispose pour cela d'un certain nombre de fonctions d'allocation mémoire et de copie, dont les plus simples sont :

- cudaError_t cudaMalloc(void **devPtr, size_t size); : alloue size octets dans la mémoire du GPU
- cudaError_t cudaFree(void *devPtr); : libère le pointeur alloué précédemment dans la mémoire du GPU
- cudaError_t cudaMemcpy(void *dst, const void *src, size_t size, enum cudaMemcpyKind kind); : réalise le transfert de l'host vers le device ou du device vers l'host

  On notera que les fonctions CUDA retournent toutes un code d'erreur.

  Seuls les kernels sont de type sous-programme (void) et ne retournent aucune valeur.

2.3. Premier programme CUDA

2.3.1. Extraction le kernel

Le kernel est le code qui sera exécuté en parallèle sur le GPU par chaque thread.

Nous allons commencer par écrire un premier programme très simple afin de présenter les concepts de base de CUDA. On désire réaliser la **somme de deux vecteurs de float** et placer le résultat de cette somme dans un troisième vecteur. Le code en langage C ressemble à cela :

- CODE

```
// compute z[i] = x[i] + y[i] for i in [0..size-1]
void sum(float *x, float *y, float *z, int size) {
  for (int i = 0; i < size; ++i) {
  z[i] = x[i] + y[i]; // paralell part
  }

}
```

Ce traitement est hautement parallèle puisque chaque somme z[i] = x[i] + y[i] est indépendante des autres, on peut donc les exécuter en parallèle.

Cette partie de code parallèle appelée **kernel** en CUDA sera exécutée par plusieurs threads et sera codée comme suit :

- CODE

```
__global__
void kernel_sum(float *x, float *y, float *z, int size) {

    /* global thread index (depends on grid organization) */
    int gtid =  ...;

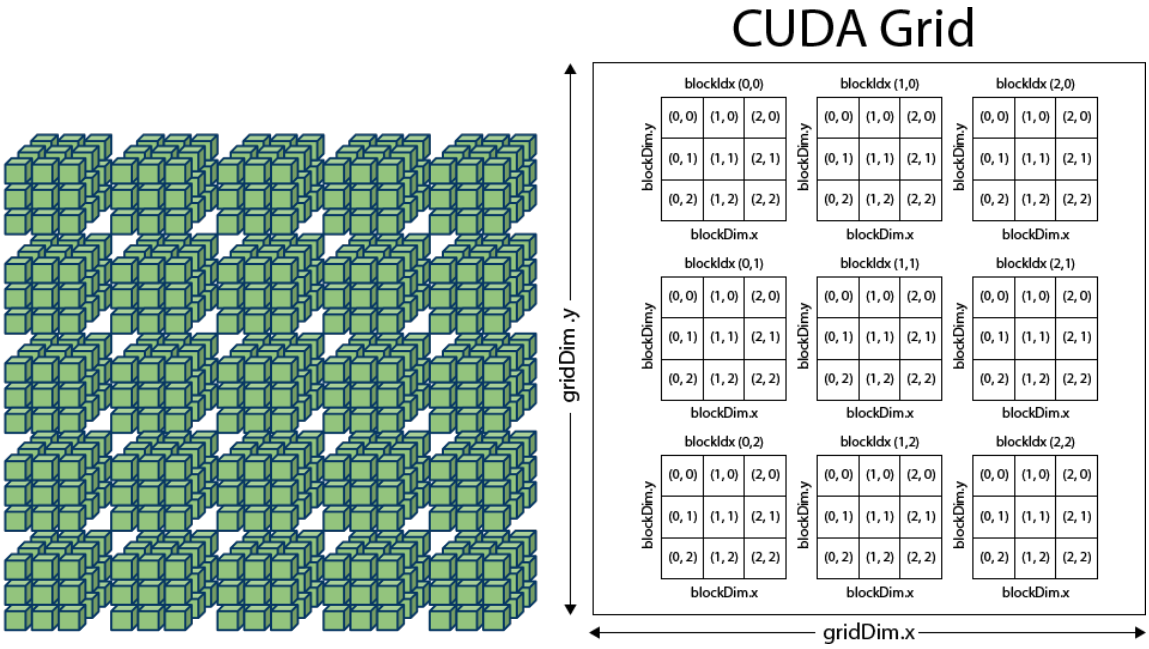
    /* check if we are in the arrays x, y, z */
    if (gtid < size) { z[gtid] = x[gtid] +
    y[gtid]; // paralell part
    }
  }
```

Plusieurs préfixes de fonctions ont été ajoutés pour permettre au compilateur CUDA (**nvcc**) de savoir quelle fonction doit être codée et executée sur **host** ou **device** et donc comment elle doit être compilée :

- **__host__ :** la fonction est exécutée par le CPU, il s'agit des fonctions du C, par défaut toute fonction est de type host
- **__global__ :** il s'agit d'une fonction exécutée par le GPU mais qui est appelée par le CPU (un **kernel**)
- **__device__ :** il s'agit d'une sous fonction exécutée par le GPU appelée depuis un kernel

2.3.2. Organisation des threads

Si on traite des vecteurs de taille *size* alors il faut pouvoir indiquer au kernel qu'on utilisera *size* threads.



L'organisation des threads est réalisée sous la forme d'une grille de blocs de threads.

- la grille est un tableau de dimension 1, 2 ou 3
- chaque élément de la grille contient un bloc de dimension 1, 2 ou 3 de threads

On dispose d'une structure de données de type dim3 qui permet de définir la taille de la grille et du bloc :

struct dim3 {int x, y, z; }

On utilisera une variable de type dim3 pour définir la taille de la grille et une autre variable de type dim3 pour définir la taille du bloc.

2.3.2.a Exemple

On désire utiliser **1023** threads, plusieurs possibilités nous sont offertes :

- grille d'un bloc composé de 1023 threads : dim3 grid(1,1,1); dim3 block(1023,1,1);

Pour les cas suivants il faudra faire en sorte que le dernier thread n'exécute pas de traitement :

- grille d'un bloc composé de 1024 threads : dim3 grid(1,1,1); dim3 block(1024,1,1);
- grille d'un bloc composé de 2 x 512 threads : dim3 grid(1,1,1); dim3 block(512,2,1);
- grille de 2 blocs composés de 512 threads : dim3 grid(2,1,1); dim3 block(512,1,1);
- grille de 4 blocs composés de 256 threads : dim3 grid(4,1,1); dim3 block(256,1,1);
- grille de 2 x 2 blocs composés de 256 threads : dim3 grid(2,2,1); dim3 block(256,1,1);
- grille de 4 x 4 blocs composés de 64 threads : dim3 grid(4,4,1); dim3 block(64,1,1);
- grille de 2 x 2 x 4 blocs composés de 2 x 32 threads : dim3 grid(2,2,4); dim3 block(32,2,1);
- ...
- grille de 2 x 2 x 4 blocs composés de 2 x 4 x 8 threads : dim3 grid(2,2,4); dim3 block(2,4,8);

Le problème majeur est que la formule qui permet d'obtenir le **global thread index** est différente en fonction de la taille 1D, 2D ou 3D de la grille et du bloc.

Une contrainte supplémentaire est liée aux caractéristiques de la carte que l'on appelle **Compute Capability** : certaines cartes acceptent un maximum de 512, 1024 ou 2048 threads par bloc.

Il faut donc faire en sorte que :

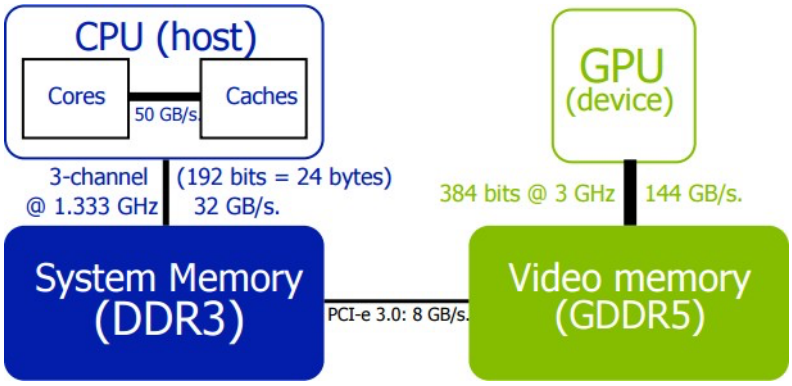
$block.x \times block.y \times block.z \leq 512$

ou 1024 ou 2048 selon les cartes.

2.3.3. Le code

Les différentes étapes à réaliser sont les suivantes

- allocation des données en mémoire centrale (cpu/host)
- allocation dans la mémoire du GPU (device)
- copie des données en entrée de host (cpu) vers device (gpu) : vecteurs x et y
- exécution du kernel copie des données en sortie de device (gpu) vers host (cpu) : vecteur z
- libération de la mémoire sur le GPU
- libération de la mémoire centrale



- CODE

```
// file sum.cu

#include <cuda.h>
#include <algorithm>
#include <cstdlib>
using namespace std;

// ====================================
// kernel declaration
// ====================================
__global__
void sum(float *x, float *y, float *z, int size) {
  // compute Global Thread InDex
  int gtid = threadIdx.x ;

  if (gtid < size) {
  z[gtid] = x[gtid] + y[gtid]; // paralell part
  }
}

// ====================================
// main function
// ====================================
int main() {
  const int SIZE = 512;

  // allocate data on computer global memory
  float *x_cpu = new float[ SIZE ];
  float *y_cpu = new float[ SIZE ];
```

```
    float *z_cpu = new float[ SIZE ];

    // fill x, y
    std::fill(&x_cpu[0], &x_cpu[SIZE], 1);
    std::fill(&y_cpu[0], &y_cpu[SIZE], 2);
    // allocate data on GPU global memory
    float *x_gpu, *y_gpu, *z_gpu;

    cudaMalloc( (void**) &x_gpu, SIZE * sizeof(float) );
    cudaMalloc( (void**) &y_gpu, SIZE * sizeof(float) );
    cudaMalloc( (void**) &z_gpu, SIZE * sizeof(float) );

    // copy x and y resp. to dev_x, dev_y
    cudaMemcpy(x_gpu, x_cpu, SIZE * sizeof(float),
                                          cudaMemcpyHostToDevice);
    cudaMemcpy(y_gpu, y_cpu, SIZE * sizeof(float),
                                          cudaMemcpyHostToDevice);

    // define number of threads
    dim3 grid(1, 1, 1);
    dim3 block(SIZE, 1, 1);
    // call kernel
    sum<<< grid, block >>>(x_gpu, y_gpu, z_gpu, SIZE);

    // copy result back to computer's global memory
    cudaMemcpy(z_cpu, z_gpu, SIZE * sizeof(float),
                                          cudaMemcpyDeviceToHost);

    // free memory
    cudaFree( x_gpu );
    cudaFree( y_gpu );
    cudaFree( z_gpu );

    delete [] x_cpu;
    delete [] y_cpu;
    delete [] z_cpu;

    exit(EXIT_FAILURE);
}
```

Dans le code précédent, l'appel au kernel est réalisé en passant le nombre de blocs dans une grille (1) ainsi que le nombre de threads par bloc, ici égal à la taille des tableaux (soit 512 éléments).

Les différentes étapes à réaliser

- allocation dans la mémoire du GPU
- copie des données en entrée de host (cpu) vers device (gpu) : vecteurs x et y
- exécution du kernel
- copie des données en sortie de device (gpu) vers host (cpu) : vecteur z
- libération de la mémoire sur le GPU

sont rébarbatives. Il est intéressant de pouvoir les encapsuler dans une classe spécifique

qui réalise ces traitements de manière semi-automatique.

Attention, avec certains version de Ubuntu, il faut installer gcc/g++ 4.9 avec le CUDA Toolkit fourni et compiler avec l'option -D_FORCE_INLINES pour nvcc afin de pouvoir résoudre le problème avec memcpy.

Exemple 2 :

Comme exemple simple, voici deux bouts de code (un pour CPU et un pour GPU avec CUDA) qui ont le même but : incrémenter de 1 chaque case d'un tableau :

Code pour CPU :

```
void main()
{
    ...
    inc_cpu(a, N);
}
void inc_cpu(int *a, int N)
{
    int idx;
    for (idx = 0; idx < N; idx++)
    a[idx] = a[idx] + 1;
}
```

Dans ce cas, le thread principal appelle la fonction qui, dans une boucle, incrémente chaque case du tableau.

Code pour GPU :

```
void main()
{
    ...
    dim3 dimBlock (blocksize);
    dim3 dimGrid (ceil(N/(float)blocksize));
    inc_gpu<<<dimGrid, dimBlock>>>(a, N);
}
__global__ void inc_gpu(int *a, int N)
{
    int idx = blockIdx.x * blockDim.x +
        threadIdx.x;
    if (idx < N)
        a[idx] = a[idx] + 1;
}
```

Ici, on déclare un kernel qui, en premier lieu, calcule son indice de thread dans sa grille (indice du bloc dans la grille x taille des blocs + indice du thread dans son bloc).

Ensuite, il se sert de cet indice pour incrémenter la case correspondante du tableau.

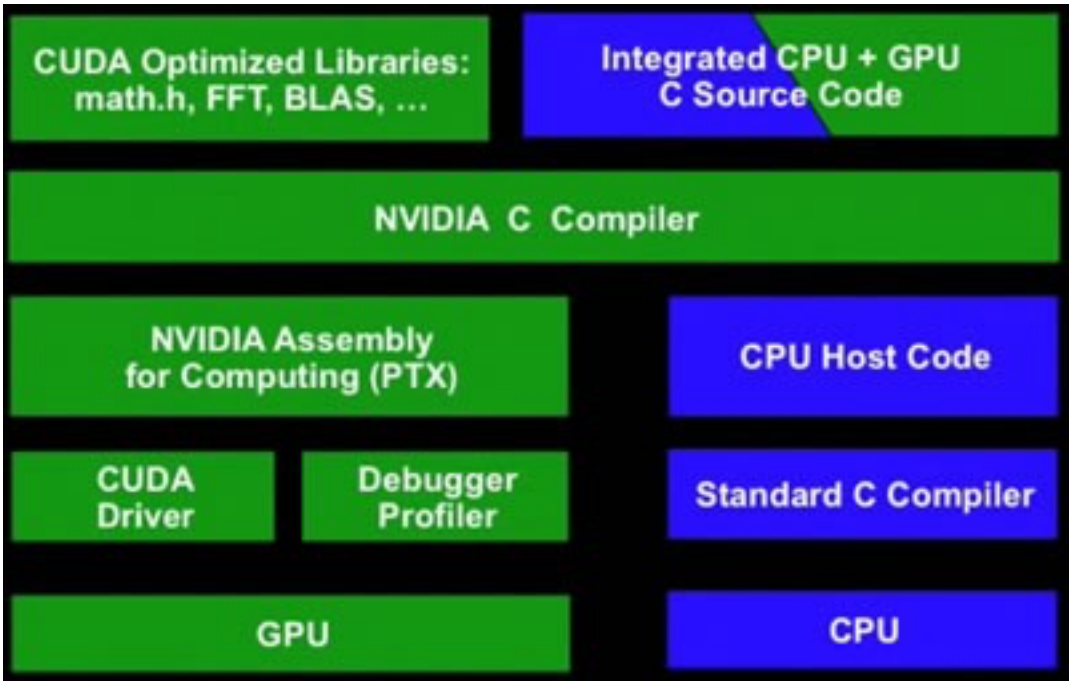
Ainsi, dans le thread principal, on lance l'exécution du kernel avec autant de threads que de case du tableau à incrémenter.

À noter, les variables blockIdx, blockDim et threadIdx sont fournies par CUDA au kernel, et dépendent des valeurs données lors de son lancement. Ces variables disposent d'attributs x, y et z pour faciliter leur repérage dans l'espace, mais ici nous n'utilisons que l'attribut x car nous avons spécifié qu'une seule dimension lors de la déclaration des variables dimBlock et dimGrid de type dim3 dans le main().

2.4. Le compilateur NVCC

CUDA fournit au programmeur un ensemble d'outils dont le compilateur **nvcc** (NVidia C Compiler). Il agit comme un compilateur C++ et est capable d'interpréter et compiler les parties de code relatives au GPU.

Le kit de développement de CUDA est composé de plusieurs éléments :



- Au sommet, le code source du programme utilisant le CPU et le GPU, et des bibliothèques optimisées fournies par nVIDIA facilitant le travail des développeurs
- En dessous, le compilateur (nvcc) prends ces fichiers en entrée et sépare le code GPU du code CPU.
- Il compile code du GPU en assembleur propriétaire (PTX) et compile le code du CPU avec GCC.
- Le code en assembleur GPU (PTX) passe, lors de l'exécution, par le pilote CUDA pour la carte graphique ou par le débugger pour etre exécuté sur le GPU
- Le code du CPU est exécuté en parallèle sur le CPU. C'est lui qui, cependant, déclenche l'exécution du code sur le GPU.

3. Le Modèle Logique

3.1. Grille et bloc

L'essentiel des performances et de l'évolutivité de CUDA réside dans le modèle logique d'exécution qui partitionne les calculs en blocs de threads et les répartit sur l'architecture hautement parallèle du GPU.

La configuration matérielle CUDA joue un rôle important sur les performances d'exécution des kernels comme nous le verrons dans le chapitre suivant.

L'écriture de fonctions parallélisables ne revêt pas de caractère complexe, par contre la répartition des threads qui exécutent les kernels en parallèle pose quelques difficultés lorsque l'on commence à apprendre CUDA.

Lorsqu'un kernel est invoqué, il s'exécute sous forme d'une grille (grid) de blocs de threads parallèles. Dans le cas de l'exemple du chapitre 2, Chaque thread exécutera la partie de code z[i] = x[i] + y[i].

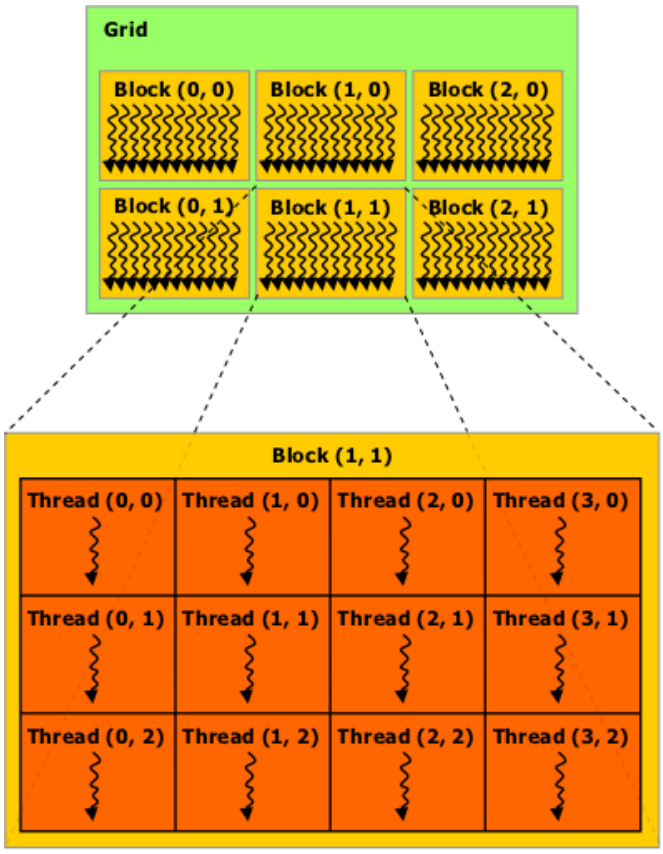
- une grille (**grid**) est un tableau à 1, 2 ou 3 dimensions (selon les caractéristiques du GPU) de blocs : grid={blocks}

- un bloc (**block**) est un tableau à 1, 2 ou 3 dimensions de threads qui peuvent partager des ressources (*shared memory*). On parle également de **CTA** (*Cooperative Threads Array*) : block={threads}

les blocs seront répartis sur les différents **SM** (*Simultaneous Processors*) pour être exécutés

Attention : le nombre maximum de threads dans un bloc est fixé par les caractéristiques de la carte (Compute Capability). Ce nombre maximum de threads sera de 512 ou 1024.

Exemple de grille 2D (y=2×x=3) avec blocs 2D (y=3×x=4)



Certaines notions sont interchangeables :

- une grille est une matrice de blocs, on peut alors soit parler de grille ou du **nombre de blocs** de la grille
- un bloc est une matrice (grille) de threads : on peut parler de taille d'un bloc ou du **nombre de threads** dans un bloc.

Afin d'éviter toute confusion nous parlerons de grille et de bloc. On parlera :

- de la grille et de sa taille (X,Y,Z) en nombre de blocs
- d'un bloc et de sa taille (X,Y,Z) en nombre de thread

Dans l'exemple précédent, on dispose d'une grille de (2 x 3) blocs. Chaque bloc se compose de (3 x 4) threads. Au final on a donc (2×3)×(3×4)=72 threads à exécuter.

Chaque thread à l'intérieur d'un bloc est identifié par une variable threadIdx (thread index) qui correspond à une structure (dim3) composée de trois champs x, y, z :

```
typedef struct {
    int x, y, z ;
```

```
} dim3;
// Variable locale au thread au sein du kernel dim3 threadIdx ;
```

Cette structure matérialise un **bloc de threads** qui peut être modélisé par une, deux ou trois dimensions. On peut en fonction des besoins n'utiliser :

- qu'une dimension : le thread sera identifié dans le bloc par la coordonnée x (threadIdx.x)
- deux dimensions : le thread sera identifié par x et y (threadIdx.y)
- trois dimensions : le thread sera identifié par x, y et z (threadIdx.z)

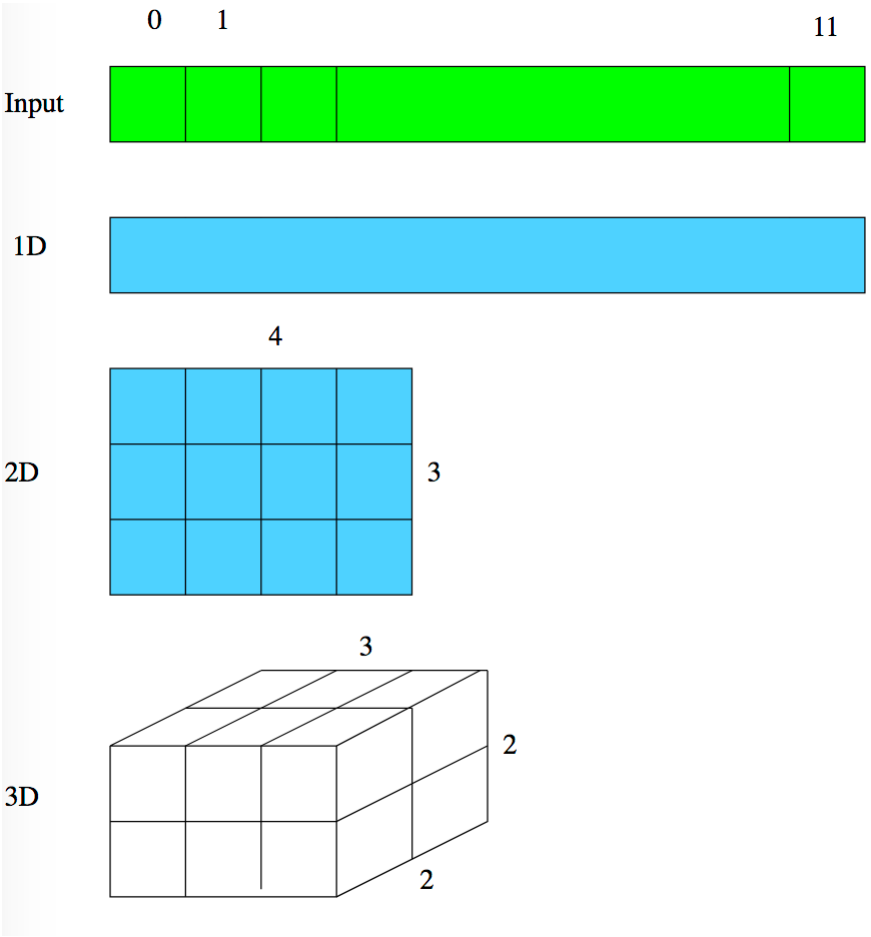
Il faudra alors établir une formule de mise en correspondance avec les indices des données en entrée (qui sont des tableaux), le tout en fonction des coordonnées x, y et z.

Attention : il n'existe pas de variable qui permet d'identifier de manière unique un thread dans l'ensemble des blocs de la grille. C'est à l'utilisateur de réaliser le calcul.

3.1.1. Exemple 1

Par exemple, si les données en entrée sont codées sous forme d'un tableau à une dimension et que le kernel est exécuté par une grille d'un seul bloc :

- pour un bloc de threads d'une seule dimension (Dx) on utilisera la coordonnée x qui correspond à l'indice **i** dans le tableau
- pour un bloc de threads à deux dimensions (Dx,Dy) l'identifiant de thread de coordonnées (x,y) sera donné par la formule : (x+y×Dx)
- pour un bloc de threads à deux dimensions (Dx,Dy,Dz) l'identifiant de thread de coordonnées (x,y,z) sera donné par la formule : (x+y×Dx+z×Dx×Dy)



3.1.2. Exemple 2

Voici un autre exemple simple qui s'appuie sur les données à traiter qui sont des tableaux à deux dimensions de float. Ici on n'a pas à établir de formule de correspondance car on établit une relation directe entre les indices de ligne et de colonne des tableaux avec respectivement y et x :

```
__global void MatAdd(float A[N][N], float B[N][N], float C[N][N])
{
        int x = threadIdx.x;
        int y = threadIdx.y;
        C[y][x] = A[y][x] + B[y][x];
}
int main()
{   ...
        // use a block of N * N threads i.e. a grid of size (1,1,1) of (N,N,1) threads
         dim3 grid(1,1,1);
        dim3 block(N, N);
        MatAdd<<< grid, block >>>(A, B, C);
}
```

Remarque : on aurait pu également utiliser des entiers lors de l'appel :

```
MatAdd <<< 1, N*N >>>(...)
```

Cependant, ici, on aura pas un bloc de dimension 2D (N,N) mais un bloc 1D de taille N*N ce qui ne permet pas d'établir la correspondance directe avec i et j. Il faudra alors utiliser la formule:

```
int y = blockIdx.x; int x = threadIdx.x;
```

3.2. Appel d'un kernel

L'appel du kernel est réalisé grâce à l'utilisation de l'opérateur d'exécution de configuration :

kernel <<< ... >>>

Cet opérateur peut prendre entre deux et quatre paramètres :

kernel<<< grid, block, shared_mem*, stream* >>> (* optionnel)

- **grid** (*dimensions de la grille*) : représente soit un entier, soit un type dim3 et permet de spécifier la dimension de la grille telle que le produit grid.x * grid.y * grid.z correspond au nombre de blocs qui seront créés. Généralement grid.z = 1
- **block** (*dimensions du bloc*) : représente soit un entier, soit un type dim3 et définit la taille d'un bloc tel que le produit block.x * block.y * block.z sera égal au nombre de threads par bloc.
- **shared_mem** (*mémoire partagée*) : entier naturel qui spécifie la taille en octets de la mémoire partagée allouée par bloc. En général égale à 0 si on a pas de traitement spécifique qui nécessite de la mémoire partagée.
- **stream** : de type cudaStream_t argument optionnel égal à 0 par défaut qui permet de spécifier un **Stream**.

Voici quelques exemples d'appel:

```
kernel<<< 10, 32 >>>(..);
kernel<<< dim3(10), 32 >>>(..);
kernel<<< dim3(10,1,1), dim3(32) >>>(..);
dim3 grid(10); dim3  block(32);
kernel<<< grid, block >>>(..);
```

L'appel à une fonction échouera si les paramètres sont erronés ou supérieurs aux tailles maximales définies par la **Compute Capability** du GPU. Il en va de même si on réserve plus de mémoire partagée que disponible.

Pour traiter, les erreurs, les fonctions CUDA retournent toutes un code d'erreur sauf l'appel au kernel puisqu'un kernel est de type void :

```
cudaError_t cudaGetLastError(void);
char* cudaGetErrorString(cudaError_t code);
```

Afin de gérer les erreurs éventuelles qui peuvent survenir il est recommandé de :

- vérifier que l'appel à chaque fonction ne produit pas d'erreur (allocation mémoire, copie des données)
- vérifier que durant l'exécution du kernel aucune erreur n'a été générée (ex: dépassement d'indice, débordement mémoire)

Pour cela on peut définir des macro-instructions en langage C. Le code suivant est issu du package CUME (CUda Made Easy)

- CODE

```
// -------------------------------------------------------------
// definition of a macro instruction that checks if a CUDA function
// was successull or not. If the execution of the function resulted
// in some error we display it and stop the program
// -------------------------------------------------------------
#define cume_check(value) {
  cudaError_t err = value;
  if (err != cudaSuccess) {
    cerr << endl;
    cerr << "=============================================\n";
    cerr << "Error: " << cudaGetErrorString(err) << " at line ";
    cerr <<_LINE     << " in file " <<  _FILE__;
    cerr <<  endl;

    exit(EXIT_FAILURE);
  }
}

// -------------------------------------------------------------
// Same as cuda_check but for kernel. This macro instruction is used
// after the execution of the kernel (see the macros KERNEL_EXECUTE_NR
// and KERNEL_EXECUTE_WR in cume_kernel.h)
// -------------------------------------------------------------
#define cume_check_kernel() {
  cudaError_t err = cudaGetLastError();
  if (err != cudaSuccess)  {
    cerr << endl;
    cerr << "=============================================\n";
    cerr << "Kernel Error: " << cudaGetErrorString(err) << " at line ";
    cerr <<_LINE     << " in file " <<  _FILE__;
    cerr <<  endl;

    exit(EXIT_FAILURE);
  }
}


// check that cudaMalloc was executed successfully
cume_check( cudaMalloc((void**) &x_gpu, SIZE * sizeof(float)) );

// check that kernel was executed successfully
sum<<< grid, block >>>(x_gpu, y_gpu, z_gpu, SIZE);
cume_check_kernel();
```

3.2.1. Notion de stream

Un Stream représente une série d'opérations à exécuter.

Le stream "0" est complétement synchrone entre host et device et agit comme si on avait inséré l'instruction cudaDeviceSynchronize() avant chaque opération à exécuter sauf si on utilise une instruction asynchrone du type cudaMemcpy*Async

3.2.1.a Exemple de traitement synchrone

```
double *cmem = new double [ size ];
cudaMalloc(&gmem, size * sizeof(double) );
// completely synchronous
cudaMemcpy(gmem, cmem, SIZE * sizeof(double), cudaMemcpyHostToDevice);
kernel1<<<grid, block>>>(gmem, ...);
kernel2<<<grid, block>>>(gmem, ...);
cudaMemcpy(cmem, gmem, SIZE * sizeof(double), cudaMemcpyDeviceToHost);
```

3.2.1.b Exemples de traitement Asynchrones

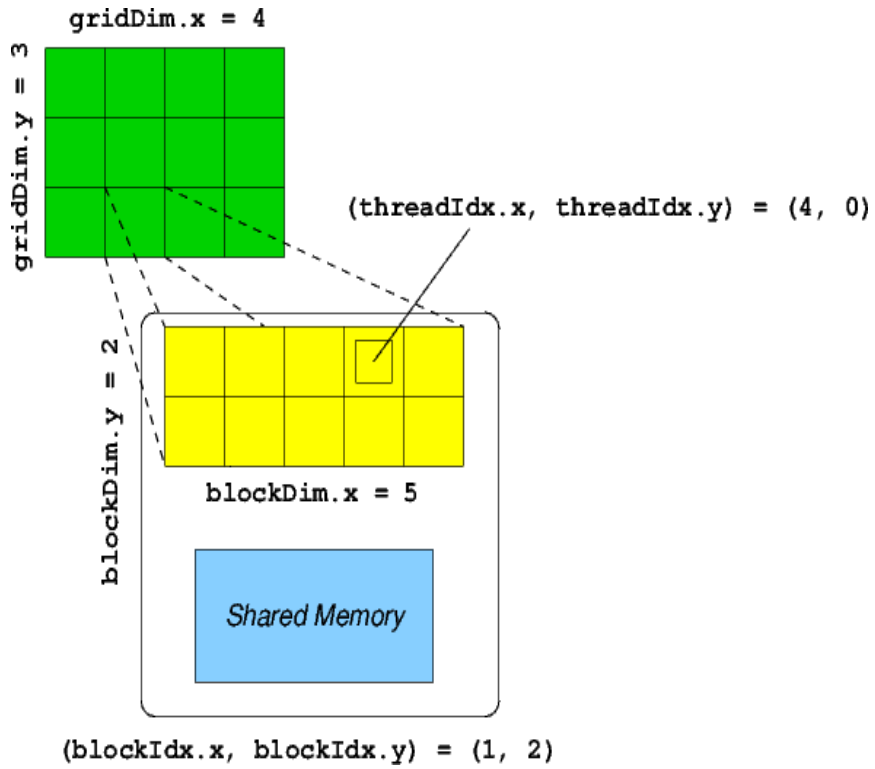
Un traitement asynchrone apparaît lorsque l'on exécute du code sur le CPU à l'intérieur d'une série d'instructions CUDA :

```
double *cmem = new double [ size ];
cudaMalloc(&gmem, size * sizeof(double) );
// completely synchronous
cudaMemcpy(gmem, cmem, SIZE * sizeof(double), cudaMemcpyHostToDevice);
kernel1<<<grid, block>>>(gmem, ...);
some_CPU_code();
kernel2<<<grid, block>>>(gmem, ...);
cudaMemcpy(cmem, gmem, SIZE * sizeof(double), cudaMemcpyDeviceToHost);
```

3.2.2. Exemple

On considère l'exemple de la figure suivante pour laquelle on dispose :

- d'une grille 2D de (3×4) blocs,
- chaque bloc 2D est composé de (2×5) threads



CUDA définit pour le kernel à exécuter des variables de type dim3 :

- gridDim : dimensions de la grille (3, 4, 1)
- blockDim : dimensions d'un bloc (2, 5, 1)
- blockIdx : coordonnées du bloc courant à l'intérieur de la grille
- threadIdx : coordonnées du thread courant à l'intérieur d'un bloc

Au final on dispose d'un ensemble de (3×4)×(2×5)=120 threads à exécuter qui seront exécutés par blocs de (2×5) threads. A l'intérieur du kernel, pour connaître les coordonnées globales d'un thread par rapport à la grille, on utilise les formules suivantes (si la grille est de type 2D et si un bloc est de type 2D) :

```
int x = blockIdx.x * blockDim.x + threadIdx.x
int y = blockIdx.y * blockDim.y + threadIdx.y
```

Si chaque thread correspond à un élément d'un tableau stocké de manière linéaire en mémoire, on détermine l'offset de l'élément par la formule :

```
int offset = y * gridDim.x * blockDim.x + x
```

Par exemple, le thread (y=1,x=3) du bloc (y=1,x=2) a pour valeurs :

x = 2 * 5 + 3 = 13 y = 1 * 2 + 1 = 3 offset = 3 * 4 * 5 + 13 = 73

Exemple d'une grille (3×4) de blocs de (2×5) threads
Dans chaque cellule apparaissent les coordonnées x et y et l'index du tableau

(0,0) 0	(0,1) 1	(0,2) 2	(0,3) 3	(0,4) 4	(0,5) 5	(0,6) 6	(0,7) 7	(0,8) 8	(0,9) 9	(0,10) 10	(0,11) 11	(0,12) 12	(0,13) 13	(0,14) 14	(0,15) 15	(0,16) 16	(0,17) 17	(0,18) 18	(0,19) 19
(1,0) 20	(1,1) 21	(1,2) 22	(1,3) 23	(1,4) 24	(1,5) 25	(1,6) 26	(1,7) 27	(1,8) 28	(1,9) 29	(1,10) 30	(1,11) 31	(1,12) 32	(1,13) 33	(1,14) 34	(1,15) 35	(1,16) 36	(1,17) 37	(1,18) 38	(1,19) 39
(2,0) 40	(2,1) 41	(2,2) 42	(2,3) 43	(2,4) 44	(2,5) 45	(2,6) 46	(2,7) 47	(2,8) 48	(2,9) 49	(2,10) 50	(2,11) 51	(2,12) 52	(2,13) 53	(2,14) 54	(2,15) 55	(2,16) 56	(2,17) 57	(2,18) 58	(2,19) 59
(3,0) 60	(3,1) 61	(3,2) 62	(3,3) 63	(3,4) 64	(3,5) 65	(3,6) 66	(3,7) 67	(3,8) 68	(3,9) 69	(3,10) 70	(3,11) 71	(3,12) 72	(3,13) 73	(3,14) 74	(3,15) 75	(3,16) 76	(3,17) 77	(3,18) 78	(3,19) 79
(4,0) 80	(4,1) 81	(4,2) 82	(4,3) 83	(4,4) 84	(4,5) 85	(4,6) 86	(4,7) 87	(4,8) 88	(4,9) 89	(4,10) 90	(4,11) 91	(4,12) 92	(4,13) 93	(4,14) 94	(4,15) 95	(4,16) 96	(4,17) 97	(4,18) 98	(4,19) 99
(5,0) 100	(5,1) 101	(5,2) 102	(5,3) 103	(5,4) 104	(5,5) 105	(5,6) 106	(5,7) 107	(5,8) 108	(5,9) 109	(5,10) 110	(5,11) 111	(5,12) 112	(5,13) 113	(5,14) 114	(5,15) 115	(5,16) 116	(5,17) 117	(5,18) 118	(5,19) 119

Comme on peut le voir sur l'exemple précédent, les threads dans un bloc accèdent des éléments d'un tableau de manière linéaire pour chaque ligne. On verra qu'il est plus intéressant d'accéder des éléments consécutifs pour chaque bloc de threads, sinon on risque de dégrader les performances.

3.3. Etude de cas

3.3.1. Somme de longs vecteurs

Considérons le cas de la somme de longs vecteurs 1D. Quel doit être le choix de la taille de la grille ou la taille d'un bloc ?

3.3.1.a solution fixe avec un seul bloc + stride

Un première solution consiste à utiliser une grille composée d'un seul bloc, le bloc étant composé de plusieurs threads et on itère jusqu'à atteindre la taille du vecteur en se déplaçant du nombre de threads (=**stride**, en français foulée, enjambée, avancée).

```
const int VECTOR_SIZE = 20480; // = 512 * 40
const int NUMBER_OF_THREADS = 512;

__global__void kernel_sum(float *x, float *y, float *z, int size)
{
    int tid = threadIdx.x;
    while (tid < size)
    {
        z[tid] = x[tid] + y[tid];
        // stride of blockDim.x
        tid += blockDim.x;
    }
}
kernel_sum<<< 1, NUMBER_OF_THREADS >>>(x_d, y_d, z_d, VECTOR_SIZE);
```

Pour cette solution qui n'utilise que 512 threads:

- le thread 0 traitera les éléments du tableau d'indice : 0, 512, 1024, ...
- le thread 1 traitera les éléments du tableau d'indice : 1, 513, 1025, ...
- le thread i traitera les éléments du tableau d'indice : i, 512+i, 1024+i, ...
- le thread 511 traitera les éléments du tableau d'indice : 511, 1023, 1535, ...

Temps d'exécution pour la somme de vecteurs en fonction de la taille du bloc

threads	time (ms)	occupancy
16	37536.0	0.021
32	31115.9	0.021
64	15745.8	0.042
128	8002.9	0.083
256	4161.2	0.167
512	2118.1	0.333
1024	1165.3	0.667

On note que pour cette solution l'occupancy est faible, il en résulte des temps de calcul élevés par rapport à la solution suivante.

3.3.1.b solution adaptative avec plusieurs blocs

Une deuxième approche consiste à adapter la taille de la grille ou du bloc à la taille du vecteur en utilisant autant de blocs que nécessaire. Dans ce cas le nombre total de threads donnés par la taille de la grille et du bloc doit être supérieur ou égal au nombre de données à traiter.

Pour un vecteur de taille 20480 (20 * 1024) floats :

- si la taille maximale d'un bloc (sur la dimension x) est de 512, alors on a 40 (= 20480 / 512) blocs dans la grille
- si la taille maximale d'un bloc (sur la dimension x) est de 1024, alors on a 20 blocs dans la grille

```
const int VECTOR_SIZE = 20480;
const int NUMBER_OF_THREADS = 512;
int NUMBER_OF_BLOCKS = (VECTOR_SIZE + NUMBER_OF_THREADS - 1) /
NUMBER_OF_THREADS;

__global__void kernel_sum(float *x, float *y, float *z, int size)
{       int gtid = threadIdx.x + blockIdx.x * blockDim.x;
        z[gtid] = x[gtid] + y[gtid];
}

kernel_sum<<< NUMBER_OF_BLOCKS, NUMBER_OF_THREADS >>>(x_d, y_d,
z_d, VECTOR_SIZE);
```

Quelle solution est la plus intéressante ? Vaut-il mieux avoir peu ou beaucoup de blocs ? Dans le cas présent, on pourrait choisir de prendre des blocs de 32 threads, ce qui donnerait une grille de 640 blocs.

Que faire si la taille de la grille n'est pas un multiple de 16, 32, ... ?

Voici quelques résultats sur **GeForce GTX 560 Ti** (10.000 itérations pour vecteur de taille 262.144 floats) :

Temps d'exécution pour la somme de vecteurs en fonction de la taille du bloc

threads	time (ms)	occupancy
16	1333.7	0.167
32	694.1	0.167
64	404.5	0.333
128	295.9	0.667
256	278.6	1.000
288	265.9	0.938
384	262.8	1.000
512	280.6	1.000
1024	369.6	0.667

On note que les temps les plus intéressants correspondent à une **occupancy** généralement maximum mais pas forcément.

3.3.1.c Nombre de threads important mais inférieurs au nombre de données à traiter

Enfin une troisième solution, similaire à la première consiste à disposer de nombreux threads (plus de 1024) mais moins que de données à traiter et à réaliser un stride.

```
const int VECTOR_SIZE = 20480;
const int NUMBER_OF_THREADS = 512;
int NUMBER_OF_BLOCKS = 4;
__global__ void kernel_sum(float *x, float *y, float *z, int size)
{
    int gtid = threadIdx.x + blockIdx.x * blockDim.x;
    int stride = gridDim.x * blockDim.x;
    while (gtid < size)
    {
        z[gtid] = x[gtid] + y[gtid];
        gtid += stride;
    }
}
```

```
kernel_sum<<< NUMBER_OF_BLOCKS, NUMBER_OF_THREADS >>>(x_d, y_d, z_d, VECTOR_SIZE);
```

Dans cet exemple on génère 4×512=2048 threads, le stride est donc de 2048, on aura donc 10 itérations de la boucle while.

3.3.1.d Mesure des performances

Les mesures de performance des deux exemples précédents sont réalisées grâce à l'API CUDA.

CUDA fournit des compteurs ainsi que des fonctions associées pour gérer ces compteurs.

- CODE

```
// definition CUDA : typedef struct CUevent_st* cudaEvent_t;

cudaEvent_t  start, stop;
cudaEventCreate(&start);
cudaEventCreate(&stop);

cudaEventRecord(start, 0); // current stream

/* ....appel kernel. ... */

cudaEventRecord(stop, 0);
cudaEventSynchronize(stop);

float elapsedTime; // temp écoulé
cudaEventElapsedTime(&elapsedTime, start, stop);
printf("elapsed: %3.1f ms\n", elapsedTime);

cudaEventDestroy(start);

cudaEventDestroy(stop);
```

L'appel à cudaEventCreate réalise une allocation mémoire d'une structure de données, il faudra donc libérer l'espace alloué grâce à cudaEventDestroy.

On peut également utiliser l'utilitaire nvprof :

nvprof --unified-memory-profiling off *program arguments*

ou son équivalent graphique nvvp (NVidia Visual Profiler).

Penser à utiliser également --print-gpu-trace ou --metrics achieved_occupancy

Lien : https://tcuvelier.developpez.com/tutoriels/gpgpu/cuda/introduction/?page=theorie#LII